

02Intro-to-Python

October 5, 2021

1 Data Science: Bridging Principles and Practice

1.1 Part 2: Introduction to Python

In this notebook, we'll cover some basics of how to send instructions to your computer in the Python programming language.

1.1.1 Table of Contents

The Data: Rocket Fuel ad campaign

[The Python Programming Language](#section 2)

[Expressions](#) and [Errors](#)

[Names](#)

[Functions](#)

[Sequences](#)

1.2 The Data: Rocket Fuel Ad Campaign

[Rocket Fuel Inc.](#) (NASDAQ: FUEL), works in digital advertising offering a "Programmatic Marketing Platform" that claims to optimize digital marketing through big data and machine learning techniques.

In 2015, Rocket Fuel ran a trial ad campaign for handbag manufacturer TaskBella. TaskBella was interested in answering two questions:

1. Would the campaign be successful?
2. If the campaign was successful, how much of that success could be attributed to the ads?

With the second question in mind, they agreed to run an **A/B test**. The majority of the people exposed to Rocket Fuel's content delivery network would see TaskBella's handbag ad (the **experimental group**). But, a small portion of people (the **control group**) would instead see a Public

Service Announcement (PSA) in the exact size and place the ad would normally be. One PSA example is below:

We can duplicate the Rocket Fuel analysis in a Jupyter notebook. But first, we need to learn a bit about how to talk to a computer.

Before we begin, we'll need a few extra tools to conduct our analysis. Run the next cell to load some code packages that we'll use later. A **code package** is a collection of code, already written by others, that we can use to analyze data.

Note: the next cell **MUST** be run in order for most of the rest of the notebook to work.

```
In [2]: # dependencies: THIS CELL MUST BE RUN
import pandas as pd
import numpy as np
import math
%matplotlib inline
from gofer.ok import check
```

DataFrames are fundamental ways of organizing and displaying data in tables. Run the next cell to load the Rocket Fuel case data into a DataFrame.

```
In [3]: # run this cell
ads = pd.read_csv('data/rocketfuel_data_renamed.csv', index_col=0)

# display the first five rows
ads.head()
```

```
Out[3]:
```

	user id	test group	converted	total ads	most ads day	most ads hour
0	1069124	ad	False	130	Monday	20
1	1119715	ad	False	93	Tuesday	22
2	1144181	ad	False	21	Tuesday	18
3	1435133	ad	False	355	Tuesday	10
4	1015700	ad	False	276	Friday	14

This table, which we've named `ads`, is organized into six **columns**: one for each *category* of information collected about each user:

user id	test group	converted	total ads	most ads day	most ads hour
The unique identifier for that user	Which testing group the user was in: "ad"- where users saw the ads (the experimental group) or "psa"- where users saw the PSAs (the control)	Whether or not the user bought a hand-bag (True if they did, False if they didn't)	The total number of ads (or PSAs) seen by that user	The day of the week on which the user saw the most ads (or PSAs)	The hour of the day during which the user saw the most ads (or PSAs)

You can also think about the table in terms of its **rows**. Each row represents all the experimental information collected about a particular user. By default only the first five rows are shown. Can you see how many rows there are in total?

The data in ads broadly falls into two types: numbers and text. *Numerical data* shows up green in code cells and can be positive, negative, or include a decimal.

```
In [4]: # Numerical data
```

```
4
87623000983
-667
3.14159
```

```
Out[4]: 3.14159
```

Text data (also called *strings*) shows up red in code cells. Strings are enclosed in double or single quotes. Note that numbers can appear in strings.

```
In [5]: # Strings
```

```
"a"

"Hi there!"

"We hold these truths to be self-evident, that all men are created equal."

# to the computer this is a string, NOT numerical data
"3.14159"
```

```
Out[5]: '3.14159'
```

Finally, there's one special kind of data that looks like text at first glance. *Boolean* data are truth values- they say whether or not a condition or state is true. There are only two possible Booleans, True and False.

Booleans show up as bolded green text in code cells.

```
In [6]: # Boolean data
```

```
True

False
```

```
Out[6]: False
```

QUESTION: Take another look at the data in the ads DataFrame. Which columns have numerical data? Which columns have string data? Which column has Boolean data?

ANSWER: Type your answer here

total ads, most ads hour - Numerical test group, most ads day, user id - String converted - Boolean

2 2. Python

Python is programming language- a way for us to communicate with the computer and give it instructions. Just like any language, Python has a *vocabulary* made up of words it can understand, and a *syntax* giving the rules for how to structure communication.

Like natural human languages, Python has rules. It differs from natural language in two important ways: 1. The rules are *simple*. You can learn most of them in a few weeks and gain reasonable proficiency with the language in a few months. 2. The rules are *rigid*. If you're proficient in a natural language, you can understand a non-proficient speaker, glossing over small mistakes. A computer running Python code is not smart enough to do that.

In this notebook, we're going to learn a few of those rules.

2.1 2a. Expressions

A piece of communication in Python is called an **expression**- it tells the computer what to do with the data we give it.

Here's an example of an expression.

```
In [7]: # an expression
        14 + 20
```

When you run the cell, the computer **evaluates** the expression and prints the result. Note that only the last line in a code cell will ever be printed, unless you explicitly tell the computer you want to print the result.

```
In [7]: # more expressions. When you run the cell, what gets printed and what doesn't?
        100 / 10

        print(4.3 + 10.98)

        33 - 9 * (40000 + 1)

        884

15.280000000000001
```

```
Out[7]: 884
```

Many basic arithmetic operations can be used in Python, like * (multiplication), + (addition), - (subtraction), and / (division). There are many others, which you can find information about [here](#).

The computer evaluates arithmetic according to the PEMDAS order of operations (just like you may have learned in middle school): anything in parentheses is done first, followed by exponents, then multiplication and division, and finally addition and subtraction.

EXERCISE: According to the PEMDAS order of operations, what should the next cell print? Work it out in your head or on paper, then run the cell to check your answer.

```
In [8]: # before you run this cell, can you say what it should print?
        # run the cell to check your answer
        4 - 2 * (1 + 6 / 3)
```

Out [8]: -2.0

OPTIONAL: If you're new to programming, use the next cell to practice arithmetic operations on numbers using operators like * (multiplication), + (addition), - (subtraction), and / (division)

```
In [9]: # practice doing arithmetic in Python
        5 - (3/6)*2
```

Out [9]: 4.0

2.1.1 A Note on Errors

Whenever you write code, you'll make mistakes. When you run a code cell that has errors, Python will sometimes produce error messages to tell you what you did wrong.

Errors are okay; even experienced programmers make many errors. When you make an error, you just have to find the source of the problem, fix it, and move on.

We have made an error in the next cell. Run it and see what happens.

```
In [12]: print("This line is missing something.")
```

This line is missing something.

You should see something like this (minus our annotations):

The last line of the error output attempts to tell you what went wrong. The *syntax* of a language is its structure, and this `SyntaxError` tells you that you have created an illegal structure. "EOF" means "end of file," so the message is saying Python expected you to write something more (in this case, a right parenthesis) before finishing the cell.

There's a lot of terminology in programming languages, but you don't need to know it all in order to program effectively. If you see a cryptic message like this, you can often get by without fully deciphering it.

EXERCISE: Scroll back up to the cell that generated an error. Fix the error, and re-run the cell to check that the error has been fixed.

2.2 2b. Names

Sometimes, the values you work with can get cumbersome- maybe the expression that gives the value is very complicated, or maybe the value itself is long. In these cases it's useful to give the value a **name**.

We can name values using what's called an *assignment* statement.

```
In [13]: # assigns 442 to x
        x = 442
```

The assignment statement has three parts. On the left is the *name* (`x`). On the right is the *value* (442). The *equals sign* in the middle tells the computer to assign the value to the name.

You'll notice that when you run the cell with the assignment, it doesn't print anything. But, if we try to access `x` again in the future, it will have the value we assigned it.

```
In [14]: # print the value of x
        x
```

```
Out[14]: 442
```

You can also assign names to expressions. The computer will compute the expression and assign the name to the result of the computation.

```
In [15]: y = 50 * 2 + 1
        y
```

```
Out[15]: 101
```

We can then use these name as if they were numbers.

```
In [16]: x - 42
```

```
Out[16]: 400
```

```
In [17]: x + y
```

```
Out[17]: 543
```

EXERCISE: Before Rocket Fuel can evaluate the effectiveness of the ad campaign, they need to know how much it cost.

The *total number of advertisements* was 14597182. The CPM (cost per thousand ads) was \$9. Use these numbers to assign the correct values to `total_ads`, `cpm`, and `cost_per_ad`.

Note: for the third variable, we want the cost *for each ad*. What do we need to do to the CPM to get the per-ad cost?

```
In [18]: # replace the ... with the total number of ads
        total_ads = 14597182
        total_ads
```

```
Out[18]: 14597182
```

```
In [19]: # replace the ... with the cost per thousand ads
        cpm = 9
        cpm
```

```
Out[19]: 9
```

```
In [20]: # replace the ... with an expression to calculate the cost per ad
        cost_per_ad = cpm/1000
        cost_per_ad
```

```
Out[20]: 0.009
```

```
In [21]: # Run this cell to check your answer for some common errors
        check("tests/02/cpm.ok")
```

```
Out [21]: <gofer.ok.OKTestsResult at 0x7f13e4606438>
```

EXERCISE: Then, calculate the overall cost by multiplying the number of ads by how much each ad cost. Assign this value to the name `cost`.

Hint: you can do the calculation by using only using `total_ads`, `cost_per_ad`, and the `*` multiplication operator- no numbers needed. Your answer should be a six-digit number (before the decimal).

```
In [22]: # replace the ... with an expression to calculate the cost of the ad campaign
cost = total_ads * cost_per_ad
cost
```

```
Out [22]: 131374.63799999998
```

```
In [23]: # Run this cell to check your answer for common errors
check("tests/02/cost.ok")
```

```
Out [23]: <gofer.ok.OKTestsResult at 0x7f13b19f8780>
```

2.3 2c. Functions

We've seen that values can have names (often called **variables**), but operations may also have names. A named operation is called a **function**. Python has some functions built into it.

```
In [24]: # a built-in function
round
```

```
Out [24]: <function round>
```

Functions get used in *call expressions*, where a function is named and given values to operate on inside a set of parentheses. The `round` function returns the number it was given, rounded to the nearest whole number.

```
In [25]: # a call expression using round
round(1988.74699)
```

```
Out [25]: 1989
```

A function may also be called on more than one value (called *arguments*). For instance, the `min` function takes however many arguments you'd like and returns the smallest. Multiple arguments are separated by commas.

```
In [26]: min(9, -34, 0, 99)
```

```
Out [26]: -34
```

PRACTICE:

- The `abs` function takes one argument (just like `round`)
- The `max` function takes one or more arguments (just like `min`)

Try calling `abs` and `max` in the cell below. What does each function do?
Also try calling each function *incorrectly*, such as with the wrong number of arguments. What kinds of error messages do you see?

```
In [27]: # replace the ... with calls to abs and max
        abs(-5)
```

```
Out[27]: 5
```

```
In [28]: max(4, 5, 0, -100)
```

```
Out[28]: 5
```

2.3.1 Dot Notation

Python has a lot of **built-in functions** (that is, functions that are already named and defined in Python), but even more functions are stored in collections called *modules*. Earlier, we imported the `math` module so we could use it later. Once a module is imported, you can use its functions by typing the name of the module, then the name of the function you want from it, separated with a `.`

```
In [29]: # a call expression with the factorial function from the math module
        math.factorial(5)
```

```
Out[29]: 120
```

PRACTICE: `math` also has a function called `sqrt` that takes one argument and returns the square root. Call `math.sqrt` on 16 in the next cell.

```
In [30]: # use math.sqrt to get the square root of 16
        root_16 = math.sqrt(16)

        # show the result of the expression
        root_16
```

```
Out[30]: 4.0
```

2.4 2d. Sequences

Working with big data, we want to be able to work with many values at the same time rather than manipulating each data point individually. We can do this using *sequences*: collections of data, all sharing the same type (e.g. numerical).

The sequence we'll work with the most is an **array**. Arrays are made using the `np.array` function.

As an example, we might look at prices for a TaskBella handbag at different stores.

```
In [31]: # make an array
        prices = np.array([105.99, 99.99, 119.95, 130, 124.99])

        prices
```

```
Out [31]: array([105.99,  99.99, 119.95, 130.   , 124.99])
```

You can retrieve items in an array by **indexing**. To index an item, put the numerical position of the item in square brackets next to the name of the array.

```
In [32]: # get the item in position 1
        prices[1]
```

```
Out [32]: 99.99
```

When we ask for the item in position 1, we get 99.99. This is because arrays are *zero-indexed*: the index starts counting at zero. So, the first item in the array is at position 0, the second item is at position 1, and so on.

PRACTICE: Try indexing the second-to-last item from the `prices` array. Save it to the name `next_to_last_price`.

```
In [33]: # index the second-to-last item in the list
        next_to_last_price = prices[3]

        # show the second-to-last item in the array
        next_to_last_price
```

```
Out [33]: 130.0
```

```
In [34]: # Run this cell to check your answer for common errors
        check("tests/02/price-index.ok")
```

```
Out [34]: <gofer.ok.OKTestsResult at 0x7f13b19f8e48>
```

2.4.1 Element-wise operations

In some cases, we may want to do calculations on each individual item in the array to return a new array of the same length.

We can do the *same operation* on every array item using arithmetic operators. This is called an **element-wise** operation. For instance, we might want to calculate the price for 5 handbags bought at each of the different stores.

```
In [35]: # multiply each price by 5
        prices * 5
```

```
Out [35]: array([529.95, 499.95, 599.75, 650.   , 624.95])
```

We can also use operators on two arrays of the same length to operate on each pair of corresponding elements. For example, we might multiply our `prices` array by an array of tax rates for each store to get the amount of sales tax.

```
In [36]: tax_rates = np.array([0.095, 0.11, 0.087, 0.1, 0.084])

        # multiply each price by its corresponding tax
        prices * tax_rates
```

```
Out [36]: array([10.06905, 10.9989 , 10.43565, 13.   , 10.49916])
```

2.4.2 Reductions

In other cases, we might want to *reduce* an array of numbers to a single value using a particular function. Some examples of reduction functions are `sum`, `min`, `max`, `average`, and `median`. Many array functions come from the *Numpy* module. Just like with the `math` module, we can call functions from the Numpy module using dot notation. Numpy is abbreviated as `np`.

```
In [37]: # get the average handbag price
         np.average(prices)
```

```
Out[37]: 116.184
```

```
In [38]: # get the lowest sales tax rate
         np.min(tax_rates)
```

```
Out[38]: 0.084
```

EXERCISE: Use the `prices` and `tax_rates` arrays to try some operations.

replace the first ellipses with an expression to add 10 to each price.

replace the second set of ellipses with an expression to divide all taxes in half

replace the last set of ellipses to create a variable `max_price` that is the largest price

</div>

```
In [39]: # use an element-wise operation to add 10 to all the prices
         price_plus_10 = prices + 10
         price_plus_10
```

```
Out[39]: array([115.99, 109.99, 129.95, 140.   , 134.99])
```

```
In [41]: # use an element-wise operation to divide all the taxes values by 2
         taxes_reduced_by_half = tax_rates / 2
         taxes_reduced_by_half
```

```
Out[41]: array([0.0475, 0.055 , 0.0435, 0.05   , 0.042 ])
```

```
In [42]: # use the np.max reduction function to get the maximum value from the prices array
         max_price = np.max(prices)
         max_price
```

```
Out[42]: 130.0
```

```
In [43]: # Run this cell to check your answer for common errors
         check("tests/02/array-ops.ok")
```

```
Out[43]: <gofer.ok.OKTestsResult at 0x7f13d461e080>
```

2.5 Conclusion

In this notebook, we covered expressions, names, functions, and sequences in Python: what they are and how to use them. We got some practice using Python to calculate some important numbers for the Rocket Fuel analysis.

In the next notebook, we'll learn how to manipulate and transform data in a `DataFrame`.

References

- "A Note on Errors" subsection and "error" image adapted from materials by Chris Hench and Mariah Rogers for the Medieval Studies 250: Text Analysis for Graduate Medievalists [data science module](#).
- Rocket Fuel data and discussion questions adapted from materials by Zsolt Katona and Brian Bell, BerkeleyHaas Case Series

Author: Keeley Takimoto